

Date: July 23rd 2026
Author – Lochana Ragupathy
Copy Rights @ lochanaragupathy.com

VIZHI

API Monitoring & Observability Platform

System Architecture Document

Prepared as a functional & data flow architecture reference
Version 1.1

Author
Lochana Ragupathy

Table of Contents

Table of Contents2

1. Executive Summary3

 1.1 Core Value Proposition3

2. Who Vizhi Serves4

3. System Context.....5

4. Functional Flow6

5. Data Flow.....7

6. Technical Architecture8

 6.1 Technology Stack8

7. Data Ingestion: Current State and Roadmap.....9

 7.1 Current State — Excel-Based Import.....9

 7.2 Phase 1 (Planned) — Automated Capture.....9

 7.3 Phase 2 (Planned) — AI Agent: Anomaly & Leak Detection.....9

8. GenAI-Powered Next Actions10

 8.1 How it Works10

 8.2 Why This Matters10

9. Appendix: Glossary11

1. Executive Summary

Vizhi is an API monitoring and observability platform built to give engineering and business stakeholders a single, shared source of truth about the health of an organization's APIs. Rather than leaving API risk buried in logs that only developers can interpret, Vizhi surfaces it as a prioritized, role-appropriate view for developers, quality engineers, and executives.

The platform's core differentiator is that it doesn't just report problems — it tells developers what to do next. A GenAI layer, powered by Google's Gemini models (with Groq as a resilient fallback provider), continuously reads the current state of the system and produces a ranked list of the three most impactful actions an engineering team should take right now.

This document describes Vizhi's functional flow (how value moves through the system for each type of user) and its data flow (how information physically moves between components), followed by the underlying technical architecture and stack.

1.1 Core Value Proposition

- Helps engineering teams catch bugs and issues in production with significantly less manual effort.
- Gives executives a clear, non-technical picture of how their APIs are functioning across the estate.
- Helps developers resolve issues with the correct priority, instead of guessing what matters most.
- Tells developers the next best action to take — not just what is wrong, but what to do about it.

2. Who Vizhi Serves

Vizhi is deliberately built around three distinct personas, each of whom needs a different slice of the same underlying data:

Persona	What they need from Vizhi	Primary surface
Developers	Prioritized findings, root-cause context, and a clear next action per endpoint.	Security tab, Endpoint Drilldown
Quality (QA) Team	Visibility into security and data-leak findings across the full API surface, independent of who owns each endpoint.	Security tab, Usage tab
Executives	A single health score and trend, without needing to interpret raw logs or technical detail.	Overview dashboard (KPI cards)

3. System Context

At the highest level, Vizhi sits between the application being monitored (the “source app”) and the humans who need to act on its health. Today, data enters Vizhi via an Excel-based import; the same integration point is designed to be replaced by an autonomous AI Agent in a future iteration without changing anything downstream.

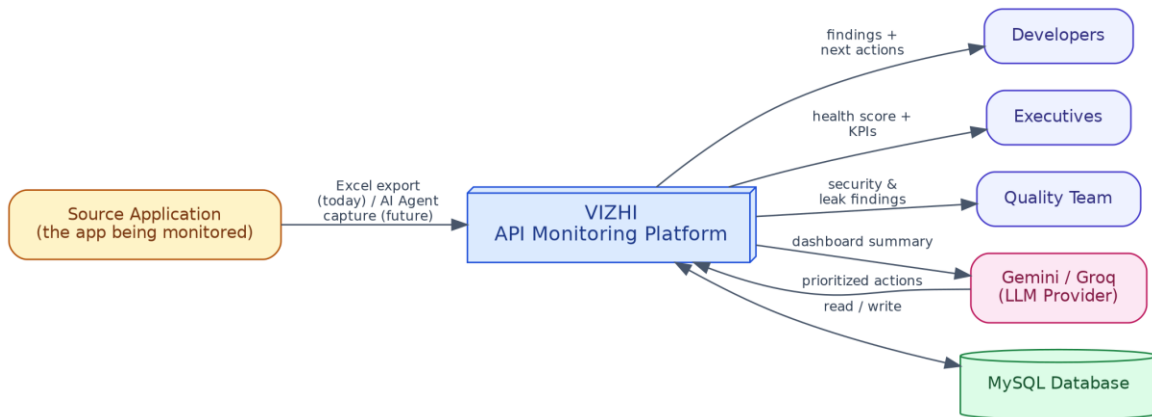


Figure 1. System Context Diagram — actors, the Vizhi platform, and its external dependencies.

4. Functional Flow

The functional flow describes how a single piece of information — an API being called in the source application — eventually becomes a resolved issue, and how that resolution feeds back into the system's understanding of overall health.

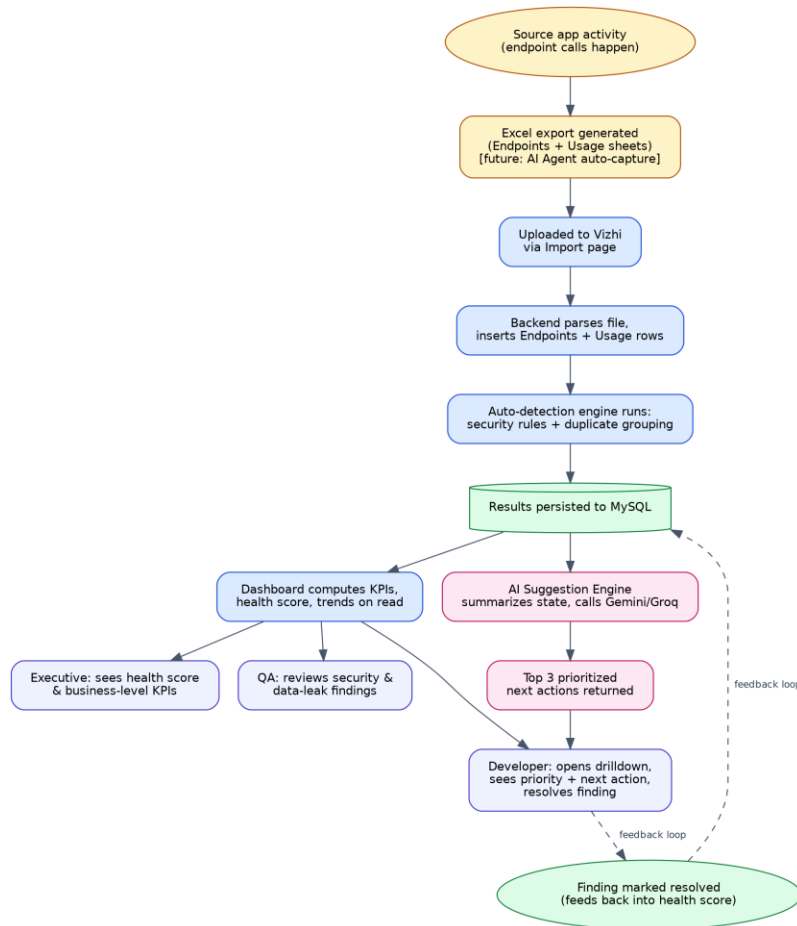


Figure 2. Functional Flow Diagram — from source-app activity to resolved finding.

Three points are worth calling out explicitly:

- Detection is automatic on import. Security rules (e.g. missing authentication, sensitive data exposed in a URL) and duplicate-endpoint grouping run immediately after data lands, with no manual triggering required.
- The AI Suggestion Engine runs independently of any single user action — it summarizes current state on demand and is cached for 10 minutes so it stays responsive without over-calling the LLM provider.
- Resolving a finding is a feedback loop, not a dead end: marking a finding resolved changes the health score on the next read, so executives and QA see the impact of developer work without any separate reporting step.

5. Data Flow

The data flow diagram traces the same journey at a system-component level — what actually moves between the Excel file, the FastAPI backend, MySQL, the external LLM provider, and the React frontend.

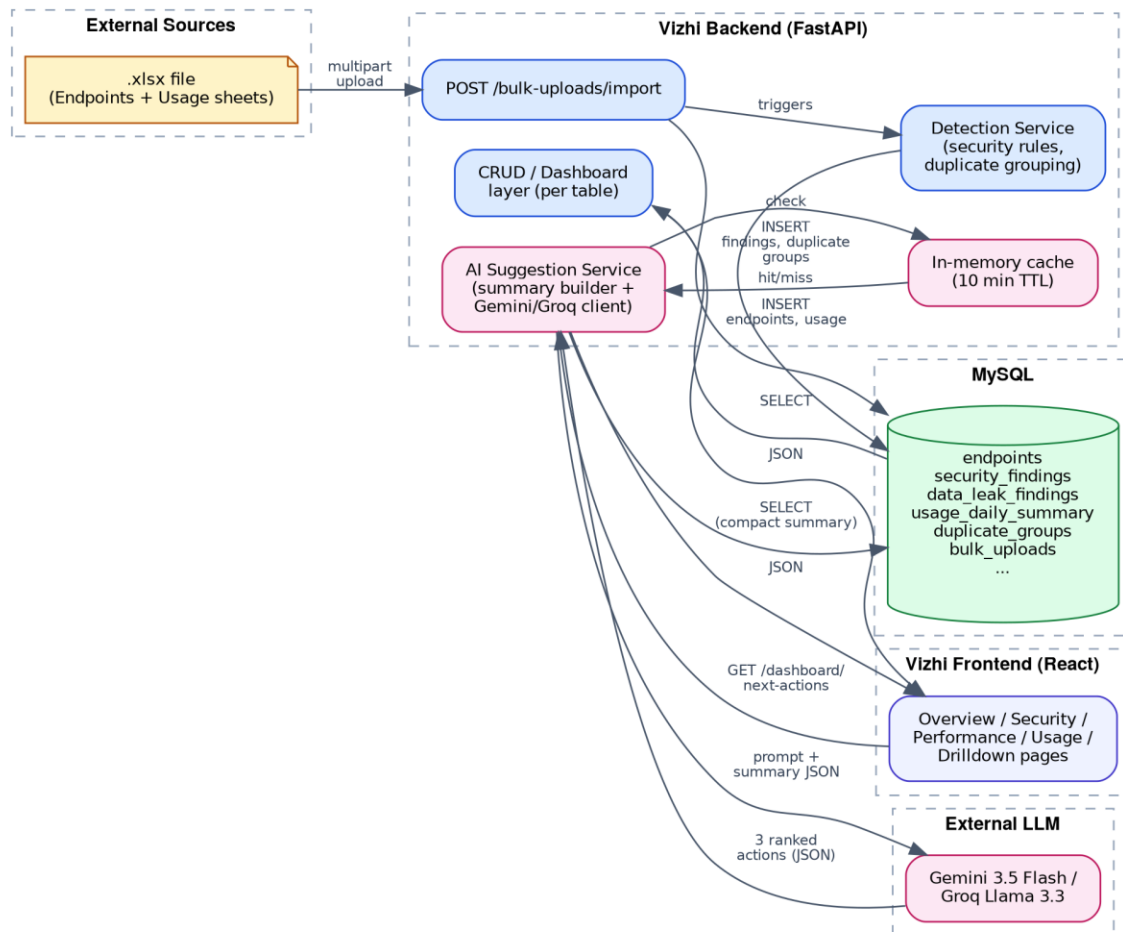


Figure 3. Data Flow Diagram — component-level movement of data through Vizhi.

Two design decisions shown in this diagram are intentional and worth documenting:

- The AI Suggestion Service never sends raw database rows to the LLM. It builds a small, purpose-built summary (KPI totals, top slow endpoints, unresolved finding counts) before the prompt is constructed — keeping the payload compact and avoiding any unnecessary exposure of underlying data.
- The in-memory cache sits between the dashboard and the LLM call specifically to protect against both cost and latency: repeated dashboard loads within a 10-minute window are served instantly without a new external API call.

6. Technical Architecture

Vizhi follows a conventional layered architecture, chosen deliberately for clarity and maintainability over premature optimization. Each layer has a single, well-defined responsibility.

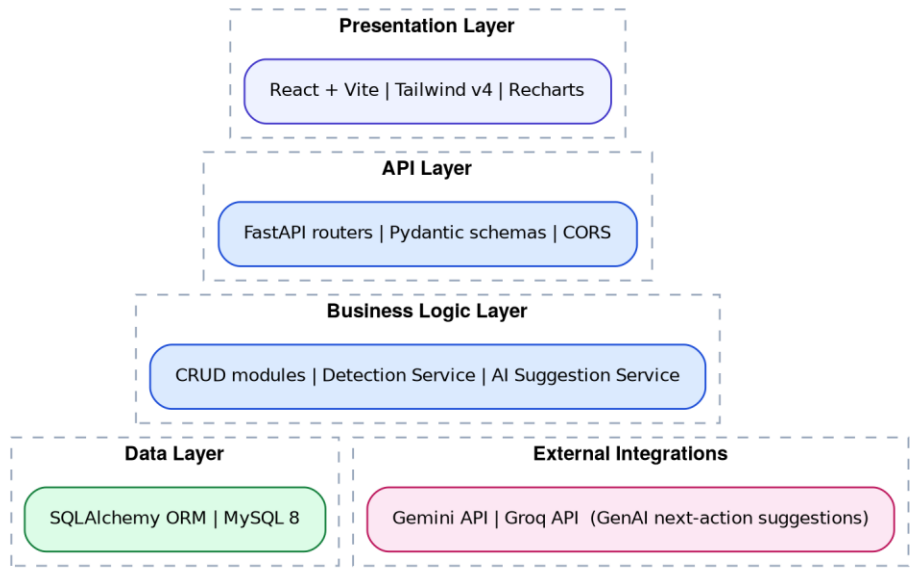


Figure 4. Layered technical architecture.

6.1 Technology Stack

Layer	Technology
Frontend	React 19, Vite, React Router, Tailwind CSS v4, Recharts, Axios
Backend API	Python, FastAPI, Pydantic (request/response validation)
Data Access	SQLAlchemy ORM
Database	MySQL 8
GenAI / LLM	Google Gemini (gemini-3.5-flash, with gemini-2.5-flash fallback); Groq (Llama 3.3 70B) as an alternate provider
Data Ingestion	Excel (.xlsx) bulk import via pandas + openpyxl (current); AI Agent-based capture (planned)

7. Data Ingestion: Current State and Roadmap

Vizhi's data ingestion layer was deliberately designed as a replaceable boundary, not a permanent constraint.

7.1 Current State — Excel-Based Import

The source application (or a capture layer sitting in front of it) exports endpoint and usage data into a two-sheet Excel workbook: one sheet listing the endpoints themselves, and one sheet with per-day usage statistics per endpoint. This file is uploaded through Vizhi's Import page, parsed, and used to populate the endpoints and usage_daily_summary tables, which in turn drive every downstream detection rule and dashboard view.

7.2 Phase 1 (Planned) — Automated Capture

The first evolution removes the manual step from today's flow, not the underlying mechanism. A scheduled capture layer collects the same endpoint and usage data currently produced by hand and pushes it directly into Vizhi on a recurring basis, using the same data contract the Excel import already relies on. This phase is deliberately non-intelligent — it automates when data arrives, without changing what Vizhi can detect.

7.3 Phase 2 (Planned) — AI Agent: Anomaly & Leak Detection

The second evolution introduces genuine reasoning into the capture layer. Rather than only forwarding traffic statistics, an AI Agent inspects live request/response pairs directly, enabling two capabilities that are structurally impossible from a static Excel snapshot: flagging likely data leaks by reasoning over actual response content (closing the data_leak_findings gap acknowledged in Section 8, which today remains manual precisely because it requires live inspection), and surfacing behavioral anomalies over time rather than waiting for a fixed threshold rule to fire. This is the point at which Vizhi's ingestion layer stops being a passive pipe and becomes an active contributor to detection.

8. GenAI-Powered Next Actions

The feature that most directly serves Vizhi's core promise — telling developers what to do next — is the AI Suggestion Engine.

8.1 How it Works

- On request, the backend assembles a compact snapshot of current system state: KPI totals, a security findings summary, the five slowest endpoints, and a count of unused endpoints.
- That snapshot is embedded in a structured prompt and sent to the configured LLM provider (Gemini by default, Groq as an alternate), which is instructed to return exactly three prioritized actions as structured JSON.
- Results are cached in memory for ten minutes, balancing freshness against cost and response latency.
- If the primary model is temporarily unavailable, the service automatically retries with backoff and falls back to an older, more stable model before surfacing an error — prioritizing a working suggestion over a perfectly fresh one.

8.2 Why This Matters

Most monitoring tools stop at reporting — they tell a team that something is wrong and leave prioritization and next steps entirely to human judgment. Vizhi's GenAI layer closes that last gap, converting raw findings into a short, actionable, business-aware to-do list every time the dashboard is opened.

9. Appendix: Glossary

Term	Definition
Health Score	Percentage of active endpoints with zero unresolved security or data-leak findings.
Finding	A detected issue on an endpoint — either a security finding (e.g. missing auth) or a data-leak finding (e.g. exposed PII).
Duplicate Group	A set of endpoints whose paths are structurally the same once variable segments (e.g. numeric IDs) are normalized.
Drilldown	The single-endpoint detail view combining usage history, findings, and duplicate membership.
Next Actions	The three LLM-generated, prioritized recommendations shown on the Overview dashboard.